UNIVERSIDAD RAFAEL LANDÍVAR

Facultad de Ingeniería

Ingeniería en Informática y Sistemas

Campus San Alberto Hurtado, Quetzaltenango

Bases de Datos II

Ing. Manuel Gustavo Rojas Vásquez

Proyecto final — Entrega II

Informe técnico del Centro Médico Pediátrico

Modelo documental, consistencia, seguridad e integración con MongoDB

Responsable del Centro Médico Pediátrico

De León Lima, Abel Alexander

Integrantes del proyecto (Fase I)

Caxaj Pú, Rocío Elizabeth

Poroj Rivera, Eddy Aroldo

De León Lima, Abel Alexander

Quetzaltenango, 31 de octubre de 2026

Versión del código documentada: rama main, commit cd34848

Índice

1. Resumen ejecutivo 3
2. Requisitos y alcance 4
2.1 Fuentes 4
2.2 Requisitos del enunciado que aplican a Pediatría 4
3. Justificación de MongoDB para el Centro Médico Pediátrico 5
4. Arquitectura 6
4.1 Monorrepositorio y enrutamiento por institución 6
4.2 Arquitectura hexagonal de Pediatría 6
4.3 Tecnologías 6
5. Modelo documental 8
5.1 Colecciones 8
5.2 Embebido frente a referenciado 8
5.3 Identificadores, fechas y valores vacíos 9
5.4 Estado real de la base (solo lectura, 31/10/2026) 9
6. Índices 10
7. Validadores $jsonSchema 11
8. Operaciones sobre la base de datos 12
9. Consistencia e integridad 14
9.1 Atomicidad por documento y filtros condicionales 14
9.2 Concurrencia 14
9.3 Auditoría sin transacciones: outbox embebido 14
9.4 Integridad referencial 14
9.5 Garantías según la configuración 14
10. Seguridad y auditoría 16
10.1 Autenticación y autorización 16
10.2 Protección de datos (clasificación de la Fase I, p. 44) 16
10.3 Bitácora de accesos 16
11. Correcciones de integridad y concurrencia 17
12. Pruebas y resultados verificados 18
13. Matriz de trazabilidad (resumen) 19
14. Limitaciones conocidas y pendientes 20
14.1 Limitaciones del módulo pediátrico 20
14.2 Pendientes grupales (no atribuibles a Pediatría) 20
15. Instalación y verificación 21

1. Resumen ejecutivo

Este informe documenta la parte individual del proyecto final correspondiente al **Centro Médico Pediátrico**, implementado con **MongoDB**. Describe el modelo documental, la consistencia, la seguridad, las operaciones sobre la base de datos, la integración con la aplicación común y las pruebas, tal como están en la rama `main` del repositorio (commit `cd34848`, 31/10/2026). Sustituye al informe anterior de Fase II, que citaba 61 pruebas y la estructura de carpetas previa al monorrepositorio.

Resultado. El módulo pediátrico funciona de extremo a extremo: interfaz React → API Express → casos de uso → adaptador MongoDB. Cubre el registro de pacientes con tutores, el mantenimiento del historial (consultas, diagnósticos, tratamientos y prescripciones), el control de crecimiento, el esquema de vacunación, el archivo por baja y la bitácora de accesos. El servidor real (MongoDB 9.0.4 standalone) tiene los validadores `$jsonSchema` de Fase II instalados; son idénticos a los del código y ningún documento los incumple.

Indicador verificado (31/10/2026, commit cd34848)	Resultado
`npm test` (plataforma + Pediatría)	**148/148** aprobadas (30 + 118), 0 fallidas, 0 omitidas
`npm run test:e2e` (Pediatría en Edge real con MongoDB desechable)	**27/27** pasos; accesibilidad básica y contraste WCAG AA sin problemas
`npm run test:e2e:vista-previa`	**12/12** pasos; consola del navegador sin errores
`npm run build`	Compilación correcta
`npm audit` (raíz y aplicación web)	0 vulnerabilidades
`npm run db:validadores` (solo lectura, base real)	4 colecciones con validador de Fase II; 0 documentos que lo incumplen

Alcance. El proyecto es grupal: cuatro instituciones con cuatro motores distintos. Este informe cubre solo el Centro Médico Pediátrico. Los otros tres backends y motores, las consultas equivalentes entre motores, el documento comparativo conjunto y el inicio de sesión común son pendientes grupales (sección 14). Hospital, Odontología y Laboratorio solo tienen hoy su interfaz en vista previa.

> **Este sistema es un prototipo académico, no un sistema listo para producción.** Los datos son ficticios. La autenticación es de desarrollo (sin contraseñas). El servidor MongoDB es standalone y no tiene transacciones multidocumento. El sistema no genera indicaciones clínicas: solo registra lo que captura el personal.

2. Requisitos y alcance

2.1 Fuentes

Clave	Documento	Uso en este informe
[P]	Enunciado del profesor: «Proyecto final», Bases de Datos II (4 páginas)	Requisitos obligatorios. Se cita por página
[F1]	Documentación de Fase I del grupo (48 páginas); sección «Centro Médico Pediátrico», p. 41–48	Diseño pediátrico de partida (modelo, operaciones y seguridad)
[F1-H]	Misma Fase I, sección del Hospital General: §8 «Catálogo de operaciones», p. 24–25	Solo como referencia de extensiones. **No** son requisitos de Pediatría

No se dispone de retroalimentación escrita del profesor sobre la Entrega I, ni de una rúbrica adicional. Por eso el requisito «modelado corregido a partir de la retroalimentación» ([P] p. 3) se declara **no verificable**, no cumplido.

2.2 Requisitos del enunciado que aplican a Pediatría

Requisito [P]	Página	Estado en Pediatría
Se evalúan las decisiones de modelado, consistencia y seguridad de cada motor; hexagonal como habilitador	p. 1	Cumplido (secciones 4–10)
Aplicación funcional y conexión cliente-servidor	p. 3	Cumplido
Arquitectura hexagonal como punto de partida común	p. 1 y 3	Cumplido
Modelado corregido según la retroalimentación de la Entrega I	p. 3	No verificable (no hay retroalimentación)
Modelo documental: decisión justificada de qué se embebe y qué se referencia	p. 4	Cumplido
Visualización ordenada sin que la interfaz cambie según el motor	p. 4	Parcial: solo hay un motor para comparar
Registro de paciente nuevo	p. 4	Cumplido
Mantenimiento: nuevas consultas, diagnósticos, tratamientos y medicamentos	p. 4	Cumplido
Desactivación o archivo por baja (traslado, fallecimiento u otra causa)	p. 4	Cumplido
Enrutamiento por usuario al motor de su institución	p. 3 y 4	Parcial: mecanismo listo; solo hay una institución real
Documento comparativo: modelado, consistencia, seguridad y justificación del motor	p. 2 y 4	Parcial: este informe es el aporte de MongoDB
Consultas funcionando de forma equivalente en los cuatro motores	p. 4	Grupal, pendiente
Cuatro instituciones con cuatro motores distintos, sin repetir	p. 1–3	Grupal, pendiente

La matriz detallada, requisito por requisito y con su evidencia, está en `instituciones/pediatria/docs/TRAZABILIDAD_REQUISITOS.md` (resumen en la sección 13).

3. Justificación de MongoDB para el Centro Médico Pediátrico

El enunciado describe al centro pediátrico como una institución que «administra historiales con seguimiento por paciente (control de crecimiento, esquema de vacunación), con consultas frecuentes y de corta duración» ([P] p. 1). La Fase I ([F1] p. 42) eligió MongoDB por esas mismas razones:

- **Acceso centrado en el paciente:** casi todas las operaciones leen o modifican el historial completo de un paciente. Un documento por paciente permite leerlo en una sola operación.
- **Consultas frecuentes y cortas:** cada atención se agrega con un único updateOne atómico ($push) sobre ese documento, sin joins ni transacciones.
- **Series históricas:** peso, talla, perímetro cefálico y dosis se acumulan como arreglos embebidos y se grafican completos.
- **Flexibilidad:** la Fase II añadió diagnósticos, tratamientos con versiones, prescripciones y trazas sin migrar el esquema.
- **Catálogos compartidos:** médicos y vacunas se referencian para no duplicarlos en cada paciente.

A cambio, MongoDB no tiene claves foráneas: la integridad referencial se verifica en los casos de uso (sección 9). Además, el servidor del proyecto es standalone y no admite transacciones multidocumento, lo que obliga a otro mecanismo para la auditoría (outbox, sección 9.3).

4. Arquitectura

4.1 Monorrepositorio y enrutamiento por institución

Las cuatro instituciones comparten **una sola aplicación web** (apps/web, React 18 + Vite 8). Cada institución tiene su propio backend y su propio motor. Al iniciar sesión, la institución del JWT (claim `institucion`) decide qué módulo carga el frontend y a qué backend se dirigen las peticiones (`instituciones/pediatria/backend/api/enrutadorInstituciones.js`). El frontend nunca elige la institución ni los permisos: los informa el backend en `GET /api/sesion`. Hoy solo está registrada la institución `centro_pediatrico` (MongoDB). Un token de otra institución recibe 403.

```
Navegador — apps/web (React): sesión, portal, permisos, cliente HTTP común
      │  carga solo el módulo de la institución que informa el backend
      ▼
instituciones/pediatria/frontend  (modulo.js, api.js, paginas/)
      │  HTTP + JWT
      ▼
API Express — enrutador por institución — casos de uso — puertos
      ▼
adaptador MongoDB — centro_pediatrico_db
```

4.2 Arquitectura hexagonal de Pediatría

Capa	Ruta actual	Responsabilidad
Dominio	`instituciones/pediatria/backend/domain/`	Reglas independientes del motor: tutor principal, fechas, dosis duplicada, archivado, enmascarado de DPI, identificador automático
Aplicación	`backend/application/`	Casos de uso: pacientes, consultas, crecimiento, vacunación, catálogos, panel y auditoría
Puertos	`backend/ports/`	Contratos `PacienteRepository`, `MedicoRepository`, `VacunaRepository`, `AuditoriaRepository` y `UnidadDeTrabajo`; `verificarRepositorios` comprueba que se cumplan
Adaptador MongoDB	`backend/adapters/mongo/`	Repositorios, unidad de trabajo (transacción u outbox), índices y validadores `$jsonSchema`
API HTTP	`backend/api/`, `app.js`, `server.js`	Rutas, validación Zod, permisos, errores comunes y enrutador por institución
Seguridad	`backend/security.js`,`backend/permisos.js`	JWT y tabla única de permisos por rol
Doble de prueba	`tests/doubles/memoria.js`	Los mismos puertos en memoria
Frontend	`instituciones/pediatria/frontend/`	Manifiesto del módulo, cliente de la API y páginas

Los casos de uso solo conocen los puertos. El mismo conjunto funciona con el adaptador MongoDB y con el doble en memoria, y las pruebas de integración comparan los dos en escenarios críticos. Para usar otro motor bastaría un adaptador que cumpla los mismos puertos; la lógica de negocio no se reescribe ([P] p. 1 y 4).

4.3 Tecnologías

Componente	Versión
Node.js / npm	24.14 / 11.11
Express · Zod · jsonwebtoken · dotenv	4.22 · 4.6.7 · 9.0 · 16.6
Driver oficial `mongodb` · servidor	6.21 · MongoDB Community 9.0.4 (standalone)
React · React Router · Vite	18.3 · 7.18 · 8.3

Componente	Versión
Pruebas	`node:test`, doble en memoria, `mongod` desechable, `playwright-core` 1.49 con Edge o Chrome instalados

5. Modelo documental

5.1 Colecciones

Colección	Rol	Campos
`pacientes`	Principal ([F1] p. 42 y 45)	`id_paciente`, `nombres`, `apellidos`, `fecha_nacimiento`, `sexo`, `estado_historial`, `tutores[]`, `consultas[]`, `controles_crecimiento[]`, `esquema_vacunacion[]`. **Fase II:** `fecha_registro`, `registrado_por`, `ultima_actualizacion`, `fecha_archivado`, `motivo_archivado`, `archivado_por`, `bitacora_pendiente[]` (outbox interna, nunca se devuelve)
`medicos`	Catálogo referenciado ([F1] p. 43)	`id_medico`, `colegiado`, `nombres`, `apellidos`, `especialidad{id_especialidad, nombre}`
`vacunas`	Catálogo referenciado ([F1] p. 43)	`id_vacuna`, `nombre`, `descripcion`
`bitacora_acceso`	Auditoría (extensión de Fase II)	`id_evento`, `id_usuario`, `rol`, `id_paciente`, `accion`, `recurso`, `referencia`, `resultado`, `fecha_hora`, `detalle`

Subdocumentos de pacientes:

- `tutores[]`: `id_tutor`, `dpi`, `nombres`, `apellidos`, `telefono`, `correo`, `direccion`, `parentesco`, `es_principal` (exactamente uno).
- `consultas[]`: `id_consulta`, `id_medico` → `medicos`, `fecha_hora`, `motivo`, `observaciones`. **Fase II:** `diagnosticos[{codigo, descripcion}]`, `tratamiento{descripcion, recomendaciones}`, `tratamientos_anteriores[]`, `prescripciones[{id_prescripcion, fecha_emision, indicaciones_generales, medicamentos[]}]` y `registrado_por`.
- `controles_crecimiento[]`: `id_control`, `id_consulta` → consulta del mismo documento, `fecha`, `peso_kg`, `talla_cm`, `perimetro_cefalico_cm` (opcional, «si corresponde»: [F1] p. 46).
- `esquema_vacunacion[]`: `id_esquema`, `id_vacuna` → `vacunas`, `numero_dosis`, `fecha_programada`, `estado`, `fecha_aplicacion`, `lote`, `id_medico` → `medicos`. **Fase II:** `motivo_cancelacion`, `programado_por`, `aplicado_por`, `cancelado_por`.

5.2 Embebido frente a referenciado

Dato	Decisión	Justificación
Tutores	Embebido	Pertenecen al paciente, son pocos (máximo 10) y se muestran con él
Consultas, con diagnósticos, tratamiento y recetas	Embebido	Se consultan siempre dentro del historial. Una consulta completa (con receta y control) es **un solo `$push` atómico**
Controles de crecimiento	Embebido	Serie histórica corta que se grafica completa; cada control apunta a una consulta del mismo documento
Esquema de vacunación	Embebido con referencias	Estado propio por paciente, modificado con el operador posicional `$`; vacuna y médico por id
Médicos y vacunas	Referenciado	Se reutilizan en muchos pacientes; embebidos obligarían a actualizar muchos documentos
Bitácora	Colección separada	Crece sin límite y se consulta por paciente, acción o fecha. Solo se embebe temporalmente un evento pendiente (outbox)

Límite de 16 MiB por documento. Una consulta con receta ocupa unos 1–2 KB: un paciente con 2 000 consultas rondaría 4 MB, suficiente para un centro pediátrico (pacientes hasta la adolescencia). Si se superara, `consultas`

pasaría a su propia colección (patrón *subset*, con las últimas N embebidas); con la arquitectura hexagonal solo cambiaría el adaptador.

5.3 Identificadores, fechas y valores vacíos

- **Identificadores:** los `id_*` siguen el patrón `^[A-Za-z0-9_-]{1,40}$`; el `_id` interno de MongoDB no se expone. Los ids embebidos se generan con prefijo y UUID (`C-…`, `CR-…`, `EV-…`, `RX-…`, `T-…`).
- **Id automático del paciente:** `P` + correlativo de hasta 9 dígitos (`P001` … `P999999999`). La forma `P` + dígitos está reservada al sistema: un `id_paciente` manual con esa forma se rechaza (400). Ver la corrección D-01 en la sección 11.
- **Fechas:** todas se guardan como BSON Date (UTC). Las fechas sin hora (nacimiento, dosis programada) van a medianoche UTC y la interfaz las muestra en UTC; las fechas con hora se muestran en hora local.
- **Valores vacíos:** los opcionales con valor `null` se eliminan antes de guardar. En las dosis pendientes, `fecha_aplicacion`, `lote` e `id_medico` se guardan explícitamente como `null`.

5.4 Estado real de la base (solo lectura, 31/10/2026)

`centro_pediatrico_db`: 4 colecciones y 104 documentos: 5 pacientes (3 activos y 2 archivados), 3 médicos, 5 vacunas y 91 eventos de bitácora. No hay referencias rotas, dosis activas duplicadas ni fechas incoherentes, y el outbox no tiene eventos pendientes. Algunos registros anteriores a la Fase II no tienen trazas (`archivado_por`, `registrado_por`, `id_evento`); el validador los admite porque esos campos son opcionales. Todos los datos son ficticios.

6. Índices

Se crean con `createIndex` (idempotente y no destructivo) al iniciar la API o con `npm run db:indices`. **Los índices únicos son esenciales**: si alguno no puede crearse, la API no empieza a atender peticiones. Los planes se comprobaron con `explain()` en modo de solo lectura.

Colección	Índice	Propósito	Uso comprobado (explain)
pacientes	`id_paciente` (único)	Identidad y referencias; impide ids repetidos incluso entre instancias	Lectura por id: EXPRESS_IXSCAN
pacientes	`tutores.dpi`	Búsqueda exacta por DPI (solo roles administrativos)	IXSCAN en búsqueda por DPI
pacientes	`estado_historial, apellidos, nombres`	Listado por estado y orden alfabético	IXSCAN + ordenamiento en memoria por el desempate `id_paciente`
pacientes	`bitacora_pendiente.id_evento` (sparse)	Localizar eventos pendientes del outbox	IXSCAN
medicos	`id_medico` (único), `colegiado` (único)	Referencias y unicidad profesional	IXSCAN
vacunas	`id_vacuna` (único)	Referencias	IXSCAN
bitacora_acceso	`id_paciente, fecha_hora`	Bitácora de un paciente, más reciente primero	IXSCAN
bitacora_acceso	`fecha_hora`	Bitácora general	IXSCAN
bitacora_acceso	`id_evento` (único parcial)	Entrega idempotente del outbox (sin duplicar eventos)	Restricción de unicidad

Volumen. Con 20 000 pacientes ficticios en un `mongod` desechable: el listado tarda 45–66 ms por página, la detección de duplicados 1 ms y el panel completo 287 ms. Mejoras posibles: incluir `id_paciente` en el índice compuesto para evitar el ordenamiento en memoria, y un índice con `fecha_nacimiento` para la detección de duplicados.

No se creó un índice único sobre nombres + apellidos + fecha de nacimiento. Dos pacientes distintos pueden compartir esos datos (homónimos, gemelos); un índice único los bloquearía para siempre. El análisis de alternativas está en `MODELO_Y_DECISIONES.md` §4.1.

7. Validadores $jsonSchema

La validación es de **dos niveles**. Zod, en la API, rechaza campos desconocidos y operadores `$…` y aplica límites de longitud y rango. El validador `$jsonSchema` de MongoDB (`backend/adapters/mongo/validadores.js`) repite las mismas restricciones de tipo, formato y enumeración, de modo que una escritura que no pase por la API tampoco puede romper el modelo.

- Instalados en las 4 colecciones con `validationLevel: strict` y `validationAction: error`. La comparación en solo lectura los encontró **idénticos** a los del código.
- `perimetro_cefalico_cm` es opcional, como en la Fase I («si corresponde»). La API lee el validador instalado y lo exigiría si se volviera a uno anterior.
- Una escritura directa inválida (por ejemplo, `sexo: "X"`, un paciente sin tutores o un peso negativo) se rechaza con el código 121, que la API traduce a `422 VALIDACION_MOTOR`.
- Herramientas seguras: `npm run db:validadores` verifica en solo lectura. Las opciones `--respaldo`, `--aplicar` y `--revertir` requieren un usuario `dbAdmin`, crean un respaldo previo y nunca modifican documentos.

Límite de JSON Schema. El validador no expresa reglas entre campos ni referencias: dos tutores principales, una dosis `Aplicada` sin lote o un `id_vacuna` inexistente pasarían una escritura **directa**. Esas reglas las hacen cumplir el dominio y los filtros de escritura, así que **la API no las permite**.

8. Operaciones sobre la base de datos

Cada operación se valida con Zod y con las reglas de dominio antes de llegar al repositorio. **Ninguna ruta elimina documentos**: el archivo es lógico ([F1] p. 47). Cada escritura comprueba `modifiedCount = 1`; si la condición del filtro ya no se cumple, responde 409 en lugar de fallar en silencio. Roles: M = médico, S = secretaría, A = administrador.

Operación	Endpoint · roles	Comando MongoDB	Condiciones y garantías
Registrar paciente	`POST /api/pacientes` · S A	`aggregate` ($match ^P\d{1,9}$, $group + $max de $toInt) + `insertOne`	Un tutor principal; identidad duplicada → 409; id único por índice; id manual P+dígitos → 400
Listar y buscar	`GET /api/pacientes` · M S A	`find` + `countDocuments`; proyección con $filter, $first, $size, $max; sort, skip, limit	Regex escapada; DPI exacto solo para S y A; archivados solo A; S sin contadores clínicos
Expediente completo	`GET /api/pacientes/:id` · M A	`findOne` con proyección (sin _id ni outbox)	Archivado solo para A; DPI enmascarado para M; lectura auditada
Datos administrativos	`GET …/:id/administrativo` · M S A	`findOne` con proyección sin datos clínicos	Clasificación de la Fase I ([F1] p. 44)
Actualizar datos	`PATCH /api/pacientes/:id` · S A	`updateOne $set` con filtro Activo + identidad previa + $not $lt nacimiento	El nacimiento no puede quedar después de registros clínicos; sin duplicar identidad
Tutores	`PATCH/POST/PUT …/tutores` · S A	`updateOne $set tutores` con filtro `tutores: previos`	Concurrencia optimista; máximo 10; un principal
Registrar consulta	`POST …/consultas` · M	`updateOne $push consultas` (y `controles_crecimiento`)	Médico existe (422); no futura ni anterior al nacimiento; id no repetido
Diagnóstico	`POST …/consultas/:c/diagnosticos` · M	`updateOne $push consultas.$.diagnosticos` con $elemMatch	Código no repetido en la consulta
Tratamiento	`PUT …/consultas/:c/tratamiento` · M	`updateOne $set consultas.$.tratamiento` + $push tratamientos_anteriores	Conserva la versión anterior; concurrencia optimista
Prescripción	`POST …/consultas/:c/prescripciones` · M	`updateOne $push consultas.$.prescripciones`	Emisión no anterior a la consulta ni futura; 1–20 medicamentos
Historial clínico	`GET …/consultas?desde=&hasta=, /prescripciones, /diagnosticos` · M A	`findOne` + filtro y orden en la aplicación	Rango ISO 8601; lectura auditada
Crecimiento	`GET/POST …/crecimiento` · M A / M	`updateOne $push controles_crecimiento`; lectura ordenada con `edad_meses`	Consulta existente; medidas > 0; perímetro opcional
Programar dosis	`POST …/vacunacion` · M	`updateOne $push` con $not $elemMatch (vacuna, dosis, estado ≠ Cancelada)	Vacuna existe; sin dosis activa repetida
Aplicar o cancelar dosis	`PATCH …/vacunacion/:e/aplicar|cancelar` · M	`updateOne $set esquema_vacunacion.$` con $elemMatch {estado: Pendiente}	Una sola aplicación incluso con dos peticiones simultáneas (una 200, otra 409)
Archivar	`PATCH …/archivar` · A	`updateOne $set estado_historial: Archivado` + fecha, motivo y autor	Sin borrado; el historial queda de solo lectura
Dosis pendientes	`GET /api/vacunacion/pendientes` · M A	`aggregate`: $match, $unwind, $match, $sort, $limit, $project	Solo historiales activos

Operación	Endpoint · roles	Comando MongoDB	Condiciones y garantías
Panel	GET /api/panel · M S A	aggregate $group ($sum, $cond, $size, $filter) + consultas recientes ($unwind, $sort, $limit)	S recibe solo dos contadores
Catálogos	GET /api/medicos, /vacunas, /especialidades · M S A; POST · A	find ordenado; insertOne (índices únicos)	Altas solo por API; sin edición ni borrado
Bitácora	GET /api/auditoria · A	find por paciente o acción, orden fecha_hora -1; eventos con insertOne; outbox con $push / $pull	Entrega idempotente por id_evento

9. Consistencia e integridad

9.1 Atomicidad por documento y filtros condicionales

Cada caso de uso clínico es **un único `updateOne` o `insertOne`** cuyo filtro lleva la condición de negocio: historial `Activo`, dosis `Pendiente`, id no repetido o fecha de nacimiento vigente. La verificación y el cambio son indivisibles en el documento del paciente, sin ventana de carrera entre lectura y escritura.

9.2 Concurrencia

- **Concurrencia optimista:** reemplazar tutores o un tratamiento solo procede si el valor no cambió desde la lectura; si cambió, responde 409.
- **Nacimiento vigente:** consultas, controles, programación y aplicación de dosis exigen en el filtro que la fecha de nacimiento siga siendo anterior. Un cambio de nacimiento simultáneo no deja fechas incompatibles (cuatro carreras deterministas probadas).
- **Identidad del paciente:** altas y ediciones con la misma identidad destino se serializan dentro de la instancia del servicio; la edición se excluye a sí misma.
- **Id automático (D-01):** dentro de una instancia, las altas toman el correlativo de una en una. Entre instancias, el índice único `id_paciente` impide repetidos y el alta se recalcula hasta 5 veces.

9.3 Auditoría sin transacciones: outbox embebido

El servidor del proyecto es **standalone**, sin transacciones multidocumento. Para que el evento de auditoría de una escritura clínica quede registrado **si, y solo si**, la escritura se confirma, el repositorio agrega el evento al arreglo `bitacora_pendiente` del propio paciente **en el mismo `updateOne`**. Después lo copia a `bitacora_acceso` de forma idempotente (índice único `id_evento`) y lo retira con `$pull`. Si la copia falla, el evento sigue en el paciente y se reenvía al arrancar la API, cada 60 s y al consultar la bitácora. Ningún evento de una escritura confirmada se pierde ni se duplica.

Con replica set, la misma `UnidadDeTrabajo` usa `session.withTransaction`: la escritura clínica y su evento se confirman o se revierten juntos. Esta ruta está **verificada con un replica set desechable**. Migrar el servidor real requiere la autorización del administrador (plan en `OPERACION_MONGODB.md`).

9.4 Integridad referencial

MongoDB no tiene claves foráneas. Los casos de uso verifican `id_medico`, `id_vacuna` e `id_consulta` antes de escribir (422 si no existen). Como no hay rutas de borrado de catálogos, no pueden quedar referencias colgantes.

9.5 Garantías según la configuración

Escenario	Standalone (servidor actual)	Replica set (verificado en desechable)
Escritura clínica + evento	Atómico en el documento; copia posterior idempotente	Transacción: ambos o ninguno
Lectura clínica con la bitácora caída	503 `AUDITORIA_NO_DISPONIBLE`, sin datos	Igual
Alta de catálogo + evento	El alta queda; el evento puede perderse (se informa en el registro)	Transacción
Doble aplicación simultánea de dosis	Una 200 y otra 409	Igual
Dos altas iguales en una instancia	Una 201 y otra 409	Igual

Escenario	Standalone (servidor actual)	Replica set (verificado en desechable)
Dos altas iguales en dos instancias	**No garantizado**: puede crear dos pacientes	No garantizado (las transacciones no lo resuelven)
Id automático entre instancias	Nunca repetido (índice único); reintento acotado	Igual

10. Seguridad y auditoría

10.1 Autenticación y autorización

- **JWT HS256** con `iss`, `aud` y `exp` obligatorios y claim `institucion = centro_pediatrico`. Se rechazan `alg: none`, tokens alterados, ajenos, vencidos o sin expiración. La API no arranca sin un `JWT_SECRET` de al menos 32 caracteres.
- **Acceso de desarrollo** (`POST /auth/dev-login`, sin contraseña): solo con `NODE_ENV` development o test, y solo desde el propio equipo (127.0.0.1 o ::1, con `trust proxy = loopback`; un `X-Forwarded-For` manipulado no lo elude). **No es un inicio de sesión real**; el inicio de sesión común es un pendiente grupal.
- **Permisos por rol** en `backend/permisos.js`: una sola tabla que usan las rutas para autorizar y `GET /api/sesion` para informar al frontend. Las pruebas comprueban 12 permisos × 3 roles.

Operación	Médico	Secretaría	Administrador
Panel	Sí	Solo contadores de pacientes	Sí
Registrar paciente, editar datos y tutores	—	Sí	Sí
Datos generales y tutores	Sí (DPI enmascarado)	Sí	Sí
Historial clínico	Activos	—	Incluidos archivados
Consultas, diagnósticos, tratamiento, recetas, crecimiento y vacunación	Lectura y escritura	—	Solo lectura
Archivar, auditoría, archivados y altas de catálogo	—	—	Sí

10.2 Protección de datos (clasificación de la Fase I, p. 44)

- Motivo, observaciones, crecimiento y vacunas (información **restringida**) solo llegan a médico y administrador. Secretaría recibe 403 en las 12 rutas clínicas y de auditoría, comprobado también con peticiones HTTP directas.
- DPI del tutor (**restringido**): completo para secretaría y administración, que lo capturan; enmascarado para el médico (solo los 4 últimos caracteres). La búsqueda por DPI es exacta y solo para roles administrativos.
- Entrada: Zod estricto (sin campos desconocidos ni operadores `$`), parámetros de ruta con patrón, JSON de 100 KB como máximo, expresiones regulares escapadas y límite de peticiones por IP.
- Errores con formato común `{ error, codigo }`; los 500 no dan detalles internos y los cuerpos de las peticiones no se registran. Cabeceras `nosniff`, `X-Frame-Options: DENY`, `no-referrer` y `Cache-Control: no-store` en la API.
- Base de datos: el usuario de la aplicación solo tiene `readWrite` sobre `centro_pediatrico_db`; las credenciales viven en `.env`, fuera del repositorio.

10.3 Bitácora de accesos

`bitacora_acceso` registra lecturas y escrituras clínicas, listados, panel, pendientes de vacunación, accesos denegados e inicios de sesión de desarrollo, con actor, rol, acción, paciente, recurso, resultado y fecha. Las lecturas solo se entregan después de persistir su evento; si la bitácora falla, responden 503 sin datos. Las trazas embebidas (`registrado_por`, `aplicado_por`, `cancelado_por`, `archivado_por`, `reemplazado_por`) conservan quién actualizó cada registro ([F1] p. 47).

11. Correcciones de integridad y concurrencia

Las auditorías técnicas del 24, 29 y 31 de octubre de 2026 encontraron defectos. Todos los corregidos tienen una prueba de regresión.

Fecha	Defecto	Corrección
24/10	El acceso de desarrollo era alcanzable desde otro equipo detrás de un proxy local	Solo responde a peticiones locales según la IP real (`trust proxy = loopback`)
24/10	Dos altas simultáneas del mismo paciente creaban dos registros	Exclusión por identidad en el servicio de pacientes
24/10	La fecha de nacimiento podía quedar después de la vacunación registrada	Regla de dominio y filtro atómico
24/10	Una receta podía ser anterior a su consulta	Misma regla temporal al crear la consulta y al agregar la receta
24/10	Las altas de médicos y vacunas no compartían unidad de trabajo con su evento	Alta y evento en la misma unidad de trabajo (transacción con replica set)
24/10	Contador de dosis pendientes con archivados; errores sin `codigo`	Mismo criterio que la lista; código estable en todos los errores
29/10	Una edición podía crear una identidad que el alta rechazaría	Exclusión compartida para altas y ediciones, excluyendo el propio registro
29/10	Un cambio de nacimiento intercalado con una escritura clínica	Filtro atómico de nacimiento vigente en cuatro operaciones
29/10	Panel, listado y pendientes no se auditaban; se respondía con la bitácora caída	Lecturas auditadas; 503 si no se puede registrar el evento
29/10	La API arrancaba aunque fallara un índice esencial	Los índices únicos son requisito de arranque
31/10	**D-01 (ALTO):** un `id_paciente` manual de 10 o más dígitos desbordaba `$toInt` y dejaba **todas** las altas automáticas en 500. Además, con 8 altas simultáneas una agotaba sus reintentos	Secuencia solo con `P` + 1–9 dígitos; forma `P` + dígitos reservada (manual → 400); secuencia agotada → 409; asignación de uno en uno por instancia. 14 pruebas nuevas, en memoria y en MongoDB desechable

12. Pruebas y resultados verificados

Ninguna prueba toca `centro_pediatrico_db`. Las de integración y de interfaz inician un proceso `mongod` propio (puerto libre, carpeta temporal, solo 127.0.0.1) y lo eliminan al terminar; las demás usan el doble en memoria. Resultados del 31/10/2026 sobre `main` (cd34848):

Archivo de pruebas (Pediatría)	Pruebas	Qué verifica
`api.test.js`	26	API completa: pacientes, tutores, consultas, crecimiento, vacunación, archivado, permisos, bitácora y contrato de errores
`mongo.integration.test.js`	26	MongoDB real desechable (standalone y replica set): validadores, rechazo de escrituras directas, flujo clínico, datos heredados, outbox con bitácora caída, carreras y transacciones
`hallazgos-auditoria.test.js`	14	Regresión de los cinco hallazgos del 24/10
`identificador-paciente.test.js`	14	D-01: ids manuales, reservados, inválidos y heredados; secuencia agotada; altas simultáneas y dos instancias; mismo resultado en el doble y en MongoDB
`integridad-auditoria.test.js`	12	Duplicados al editar, carreras de nacimiento, lecturas auditadas e índices esenciales
`seguridad.test.js`	9	JWT, DPI enmascarado, búsqueda por DPI, secretaría sin datos clínicos, errores y perímetro opcional
`dominio.test.js` · `schema.test.js`	6 · 5	Reglas de dominio, contrato de puertos y esquemas Zod
`enrutamiento.test.js` · `permisos.test.js`	3 · 3	Despacho por institución; permisos anunciados = autorización real
Total Pediatría	**118**	
Plataforma común (`tests/*.test.js`)	**30**	Contrato de módulos, portal, módulos pendientes, cliente HTTP y proxy de Vite

E2E de Pediatría (27 pasos, Edge real + API + MongoDB desechable): inicio de sesión, panel, búsqueda, registro, edición, consulta con diagnóstico, tratamiento, receta y control, gráficas de crecimiento, vacunación, archivado, bitácora, permisos (también el rechazo directo del backend), temas claro y oscuro, teclado y foco, diseño a 1920, 1366, 820 y 390 px, caída de la API, accesibilidad básica y contraste WCAG AA. **Vista previa (12 pasos):** pantallas de las instituciones pendientes sin escrituras ni peticiones de otras instituciones.

Pruebas de auditoría adicionales (31/10/2026, MongoDB desechable): volumen de 20 000 pacientes con `explain()`; autorización por HTTP directo en todas las rutas clínicas; inyección de operadores (`q[$ne]`, `$set`) rechazada con 400; token de otra institución rechazado con 403.

13. Matriz de trazabilidad (resumen)

La matriz completa con su evidencia está en `instituciones/pediatria/docs/TRAZABILIDAD_REQUISITOS.md`. Las operaciones del catálogo del Hospital General ([F1-H] p. 24–25), como buscar al paciente por documento, el historial de medicamentos, la bitácora o el rango de fechas, **no** se atribuyen a la Fase I pediátrica: aparecen en la matriz como extensiones implementadas.

Requisito	Fuente	Estado
Colección `pacientes` con `medicos` y `vacunas` de apoyo	[F1] p. 42, 45	Cumplido
Entidades y atributos de la Fase I	[F1] p. 43	Cumplido
Clasificación pública, confidencial y restringida	[F1] p. 44	Cumplido
Embebido y referenciado según la Fase I	[F1] p. 45–46	Cumplido
Seis operaciones: consulta corta, peso y talla, aplicar vacuna, historial de crecimiento, historial completo, archivar	[F1] p. 46–47	Cumplido
Estados del esquema «programadas, pendientes, aplicadas u omitidas»	[F1] p. 42	Adaptado: `Pendiente`, `Aplicada`, `Cancelada` («omitida»)
Personal autorizado, permisos por rol, registro de cambios, archivar sin eliminar	[F1] p. 47	Cumplido
Registro, mantenimiento y archivo de historiales	[P] p. 4	Cumplido
Modelo documental justificado	[P] p. 4	Cumplido
Hexagonal y cliente-servidor	[P] p. 1, 3	Cumplido
Enrutamiento por usuario y visualización independiente del motor	[P] p. 3–4	Parcial (una sola institución real)
Modelado corregido según la retroalimentación	[P] p. 3	No verificable
Cuatro motores, consultas equivalentes y documento comparativo conjunto	[P] p. 1–4	Pendiente grupal

14. Limitaciones conocidas y pendientes

14.1 Limitaciones del módulo pediátrico

1. **Sin transacciones en el servidor real** (standalone). La consistencia se basa en la atomicidad por documento y el outbox; las transacciones solo se verificaron en un replica set desechable.
2. **Unicidad de identidad entre instancias:** con varias instancias de la API, dos altas iguales simultáneas pueden crear dos pacientes. La detección tampoco normaliza dobles espacios ni tildes («Ana Maria» frente a «Ana María»).
3. **Id automático entre instancias:** nunca se repite, pero con mucha concurrencia entre procesos un alta puede responder 409 «vuelva a intentarlo». Un P999999999 cargado directamente en la base agotaría la secuencia (409 controlado).
4. **Altas de catálogo en standalone:** el evento de auditoría se registra después del alta; si la bitácora falla, el evento puede perderse.
5. **Disponibilidad:** si la bitácora no responde, las lecturas clínicas se bloquean (503). Si MongoDB no responde, la API devuelve un 500 genérico tras unos 8 s.
6. **Catálogos:** las altas de médicos y vacunas solo están en la API (administrador). No hay edición ni desactivación, y no se puede reprogramar una dosis sin cancelarla.
7. **Seguridad de prototipo:** sin inicio de sesión real ni revocación de tokens; JWT en `sessionStorage` y sin Content-Security-Policy; la lectura de la bitácora no se audita.
8. **Rendimiento:** el listado ordena en memoria por el desempate `id_paciente`; algunos filtros (rango de fechas de consultas) se aplican en la aplicación tras leer el documento.

14.2 Pendientes grupales (no atribuibles a Pediatría)

- **Asignación definitiva de motores.** La Fase I asigna PostgreSQL al Laboratorio y MySQL a Odontología ([F1] p. 40), pero también describe al Hospital «implementado en Mysql» ([F1-H] p. 25), y Firebase no aparece asignado. Eso repite MySQL y contradice el enunciado ([P] p. 1–3).
- Backends y motores de Hospital, Odontología y Laboratorio, y su registro en la aplicación común.
- Consultas equivalentes en los cuatro motores ([P] p. 4).
- Documento comparativo conjunto de los cuatro motores ([P] p. 2 y 4); este informe aporta la parte de MongoDB.
- Inicio de sesión común con usuarios reales.

15. Instalación y verificación

Requisitos: Node.js ≥ 20.19 con npm, MongoDB Community Server 9.0 y, para las pruebas E2E, Microsoft Edge o Google Chrome. Desde la carpeta del proyecto:

```
npm ci                      # dependencias del backend y de las pruebas
npm run web:ci              # dependencias de la aplicación web
Copy-Item .env.example .env # luego definir JWT_SECRET (>= 32 caracteres)
npm run build               # compila la interfaz
npm start                   # http://localhost:3001

npm test                    # 148 pruebas (bases desechables o en memoria)
npm run test:e2e            # 27 pasos en navegador real
npm run test:e2e:vista-previa  # 12 pasos
npm run db:validadores      # solo lectura sobre la base configurada
```

El procedimiento completo (opciones de base de datos, respaldo y restauración aislada) está en el `README.md` del repositorio y en `instituciones/pediatria/docs/`.